Dropout

In this notebook, you will implement dropout. Then we will ask you to train a network with batchnorm and dropout, and acheive over 55% accuracy on CIFAR-10.

CS231n has built a solid API for building these modular frameworks and training them, and we will use their very well implemented framework as opposed to "reinventing the wheel." This includes using their Solver, various utility functions, and their layer structure. This also includes nndl.fc_net, nndl.layers, and nndl.layer_utils. As in prior assignments, we thank Serena Yeung & Justin Johnson for permission to use code written for the CS 231n class (cs231n.stanford.edu).

In [1]:
```
## Import and setups

import time
import numpy as np
import matplotlib.pyplot as plt
from nndl.fc_net import *
from nndl.layers import *
from utils.data_utils import get_CIFAR10_data
from utils.gradient_check import eval_numerical_gradient, eval_numerical_
from utils.solver import Solver

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# for auto-reloading external modules
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in
%load_ext autoreload
%autoreload 2

def rel_error(x, y):
  """ returns relative error """
  return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y)))
```

In [2]:
```
# Load the (preprocessed) CIFAR10 data.

data = get_CIFAR10_data()
for k in data.keys():
  print('{}: {} '.format(k, data[k].shape))
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

Dropout forward pass

Implement the training and test time dropout forward pass, `dropout_forward`, in `nndl/layers.py`. After that, test your implementation by running the following cell.

In [3]:
```
x = np.random.randn(500, 500) + 10

for p in [0.3, 0.6, 0.75]:
  out, _ = dropout_forward(x, {'mode': 'train', 'p': p})
  out_test, _ = dropout_forward(x, {'mode': 'test', 'p': p})

  print('Running tests with p = ', p)
  print('Mean of input: ', x.mean())
  print('Mean of train-time output: ', out.mean())
  print('Mean of test-time output: ', out_test.mean())
  print('Fraction of train-time output set to zero: ', (out == 0).mean())
  print('Fraction of test-time output set to zero: ', (out_test == 0).mea
```

```
Running tests with p =  0.3
Mean of input:  9.999576473439076
Mean of train-time output:  10.050944395997103
Mean of test-time output:  9.999576473439076
Fraction of train-time output set to zero:  0.698312
Fraction of test-time output set to zero:  0.0
Running tests with p =  0.6
Mean of input:  9.999576473439076
Mean of train-time output:  9.994659403205267
Mean of test-time output:  9.999576473439076
Fraction of train-time output set to zero:  0.400252
Fraction of test-time output set to zero:  0.0
Running tests with p =  0.75
Mean of input:  9.999576473439076
Mean of train-time output:  10.015859686058297
Mean of test-time output:  9.999576473439076
Fraction of train-time output set to zero:  0.248756
Fraction of test-time output set to zero:  0.0
```

Question

1. Compare the Mean of input and Mean of train-time output. Explain the mathematical operation in your implementation that preserves the mean.

2. Observe the Fraction of test-time output set to zero. What does this value indicate about the layer's behavior during the test phase, and why is this design choice necessary?

Answer:

1. The mean of the input and the train-time output are essentially the same. The mathematical operation that preserves this mean is known as "Inverted Dropout." During training, we divide the retained activations by the keep probability p (e.g., `out = (x * mask) / p`). This scaling ensures that the expected value of the layer's output remains constant and matches the input's mean.

2. The fraction of test-time output set to zero is exactly 0.0, indicating that all neurons remain active during the test phase (no dropout is applied). This design choice is necessary because, at test time, we want to utilize the full predictive capacity of the ensemble of sub-networks we effectively trained. Since we already scaled the activations during training (via inverted dropout), we can pass the inputs directly through at test time without any further scaling or dropping.

Dropout backward pass

Implement the backward pass, `dropout_backward`, in `nndl/layers.py`. After that, test your gradients by running the following cell:

In [4]:
```
x = np.random.randn(10, 10) + 10
dout = np.random.randn(*x.shape)

dropout_param = {'mode': 'train', 'p': 0.8, 'seed': 123}
out, cache = dropout_forward(x, dropout_param)
dx = dropout_backward(dout, cache)
dx_num = eval_numerical_gradient_array(lambda xx: dropout_forward(xx, dro

print('dx relative error: ', rel_error(dx, dx_num))
```

```
dx relative error:  5.445610262616525e-11
```

Implement a fully connected neural network with dropout layers

Modify the `FullyConnectedNet()` class in `nndl/fc_net.py` to incorporate dropout. A dropout layer should be incorporated after every ReLU layer. Concretely, there shouldn't be a dropout at the output layer since there is no ReLU at the output layer. You will need to modify the class in the following areas:

(1) In the forward pass, you will need to incorporate a dropout layer after every relu layer.

(2) In the backward pass, you will need to incorporate a dropout backward pass layer.

Check your implementation by running the following code. Our W1 gradient relative error is on the order of 1e-6 (the largest of all the relative errors).

In [5]:
```
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D)
y = np.random.randint(C, size=(N,))

for dropout in [0.5, 0.75, 1.0]:
  print('Running check with dropout = ', dropout)
  model = FullyConnectedNet([H1, H2], input_dim=D, num_classes=C,
                            weight_scale=5e-2, dtype=np.float64,
                            dropout=dropout, seed=123)

  loss, grads = model.loss(X, y)
  print('Initial loss: ', loss)
```

```
  for name in sorted(grads):
    f = lambda _: model.loss(X, y)[0]
    grad_num = eval_numerical_gradient(f, model.params[name], verbose=Fal
    print('{} relative error: {}'.format(name, rel_error(grad_num, grads[
  print('\n')
```

```
Running check with dropout =  0.5
Initial loss:  2.309771209610118
W1 relative error: 2.694274363733021e-07
W2 relative error: 7.439246147919978e-08
W3 relative error: 1.910371122296728e-08
b1 relative error: 4.112891126518e-09
b2 relative error: 5.756217724722137e-10
b3 relative error: 1.3204470857080166e-10


Running check with dropout =  0.75
Initial loss:  2.306133548427975
W1 relative error: 8.72986097970181e-08
W2 relative error: 2.9777307885797295e-07
W3 relative error: 1.8832780806174298e-08
b1 relative error: 5.379486003985169e-08
b2 relative error: 3.6529949080385546e-09
b3 relative error: 9.987242764516995e-11


Running check with dropout =  1.0
Initial loss:  2.3053332250963194
W1 relative error: 1.2744095365229032e-06
W2 relative error: 4.678743300473988e-07
W3 relative error: 4.331673892536035e-08
b1 relative error: 4.0853539035931665e-08
b2 relative error: 1.951342257912746e-09
b3 relative error: 9.387142701440351e-11
```

Dropout as a regularizer

In class, we claimed that dropout acts as a regularizer by effectively bagging. To check this, we will train two small networks, one with dropout and one without dropout.

In [6]:
```
# Train two identical nets, one with dropout and one without

num_train = 500
small_data = {
  'X_train': data['X_train'][:num_train],
  'y_train': data['y_train'][:num_train],
  'X_val': data['X_val'],
  'y_val': data['y_val'],
}

solvers = {}
dropout_choices = [0.6, 1.0]
for dropout in dropout_choices:
  model = FullyConnectedNet([100, 100, 100], dropout=dropout)
```

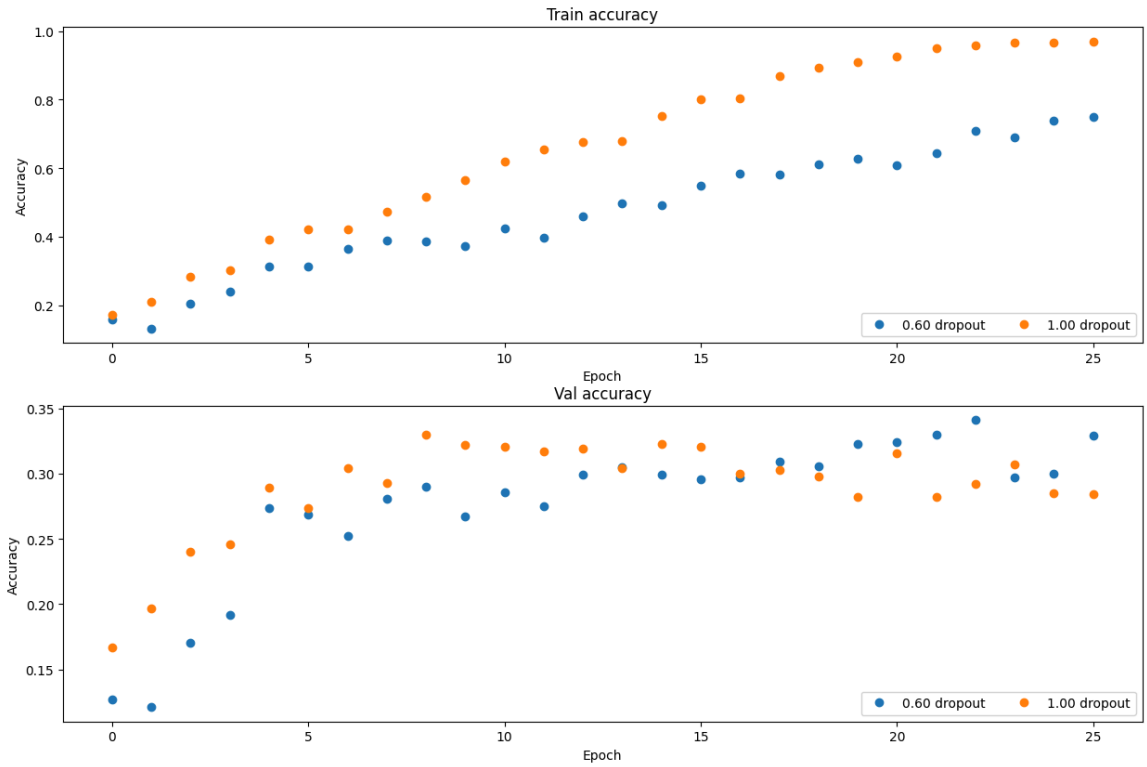
```
solver = Solver(model, small_data,
                num_epochs=25, batch_size=100,
                update_rule='adam',
                optim_config={
                  'learning_rate': 5e-4,
                },
                verbose=True, print_every=100)
solver.train()
solvers[dropout] = solver
```

```
(Iteration 1 / 125) loss: 2.300199
(Epoch 0 / 25) train acc: 0.158000; val_acc: 0.127000
(Epoch 1 / 25) train acc: 0.132000; val_acc: 0.121000
(Epoch 2 / 25) train acc: 0.204000; val_acc: 0.170000
(Epoch 3 / 25) train acc: 0.240000; val_acc: 0.192000
(Epoch 4 / 25) train acc: 0.312000; val_acc: 0.274000
(Epoch 5 / 25) train acc: 0.314000; val_acc: 0.269000
(Epoch 6 / 25) train acc: 0.364000; val_acc: 0.252000
(Epoch 7 / 25) train acc: 0.390000; val_acc: 0.281000
(Epoch 8 / 25) train acc: 0.386000; val_acc: 0.290000
(Epoch 9 / 25) train acc: 0.372000; val_acc: 0.267000
(Epoch 10 / 25) train acc: 0.424000; val_acc: 0.286000
(Epoch 11 / 25) train acc: 0.396000; val_acc: 0.275000
(Epoch 12 / 25) train acc: 0.458000; val_acc: 0.299000
(Epoch 13 / 25) train acc: 0.496000; val_acc: 0.305000
(Epoch 14 / 25) train acc: 0.492000; val_acc: 0.299000
(Epoch 15 / 25) train acc: 0.550000; val_acc: 0.296000
(Epoch 16 / 25) train acc: 0.584000; val_acc: 0.297000
(Epoch 17 / 25) train acc: 0.582000; val_acc: 0.309000
(Epoch 18 / 25) train acc: 0.612000; val_acc: 0.306000
(Epoch 19 / 25) train acc: 0.628000; val_acc: 0.323000
(Epoch 20 / 25) train acc: 0.608000; val_acc: 0.324000
(Iteration 101 / 125) loss: 1.369535
(Epoch 21 / 25) train acc: 0.644000; val_acc: 0.330000
(Epoch 22 / 25) train acc: 0.708000; val_acc: 0.341000
(Epoch 23 / 25) train acc: 0.690000; val_acc: 0.297000
(Epoch 24 / 25) train acc: 0.740000; val_acc: 0.300000
(Epoch 25 / 25) train acc: 0.750000; val_acc: 0.329000
(Iteration 1 / 125) loss: 2.300607
(Epoch 0 / 25) train acc: 0.172000; val_acc: 0.167000
(Epoch 1 / 25) train acc: 0.210000; val_acc: 0.197000
(Epoch 2 / 25) train acc: 0.284000; val_acc: 0.240000
(Epoch 3 / 25) train acc: 0.302000; val_acc: 0.246000
(Epoch 4 / 25) train acc: 0.392000; val_acc: 0.289000
(Epoch 5 / 25) train acc: 0.420000; val_acc: 0.274000
(Epoch 6 / 25) train acc: 0.420000; val_acc: 0.304000
(Epoch 7 / 25) train acc: 0.474000; val_acc: 0.293000
(Epoch 8 / 25) train acc: 0.516000; val_acc: 0.330000
(Epoch 9 / 25) train acc: 0.566000; val_acc: 0.322000
(Epoch 10 / 25) train acc: 0.620000; val_acc: 0.321000
(Epoch 11 / 25) train acc: 0.656000; val_acc: 0.317000
(Epoch 12 / 25) train acc: 0.676000; val_acc: 0.319000
(Epoch 13 / 25) train acc: 0.680000; val_acc: 0.304000
(Epoch 14 / 25) train acc: 0.752000; val_acc: 0.323000
(Epoch 15 / 25) train acc: 0.802000; val_acc: 0.321000
(Epoch 16 / 25) train acc: 0.804000; val_acc: 0.300000
(Epoch 17 / 25) train acc: 0.868000; val_acc: 0.303000
(Epoch 18 / 25) train acc: 0.894000; val_acc: 0.298000
(Epoch 19 / 25) train acc: 0.910000; val_acc: 0.282000
(Epoch 20 / 25) train acc: 0.926000; val_acc: 0.316000
(Iteration 101 / 125) loss: 0.245816
(Epoch 21 / 25) train acc: 0.950000; val_acc: 0.282000
(Epoch 22 / 25) train acc: 0.958000; val_acc: 0.292000
(Epoch 23 / 25) train acc: 0.966000; val_acc: 0.307000
(Epoch 24 / 25) train acc: 0.966000; val_acc: 0.285000
(Epoch 25 / 25) train acc: 0.970000; val_acc: 0.284000
```

In [7]:

```
# Plot train and validation accuracies of the two models

train_accs = []
```

```
val_accs = []
for dropout in dropout_choices:
  solver = solvers[dropout]
  train_accs.append(solver.train_acc_history[-1])
  val_accs.append(solver.val_acc_history[-1])

plt.subplot(3, 1, 1)
for dropout in dropout_choices:
  plt.plot(solvers[dropout].train_acc_history, 'o', label='%.2f dropout'
plt.title('Train accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(ncol=2, loc='lower right')

plt.subplot(3, 1, 2)
for dropout in dropout_choices:
  plt.plot(solvers[dropout].val_acc_history, 'o', label='%.2f dropout' %
plt.title('Val accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(ncol=2, loc='lower right')

plt.gcf().set_size_inches(15, 15)
plt.show()
```



Question

Based off the results of this experiment, is dropout performing regularization? Explain your answer.

Answer:

Yes, dropout is clearly performing regularization. In the plotted results, the model without dropout (`1.00 dropout` , meaning 100% keep probability) achieves near-perfect training accuracy, indicating that it is heavily overfitting and memorizing the training data. However, its validation accuracy is noticeably lower. In contrast, the model with dropout (`0.60 dropout`) has a lower training accuracy but achieves a higher validation accuracy than the baseline model. By shrinking the gap between training and validation accuracy, dropout successfully prevents overfitting and improves the model's ability to generalize to unseen data.

Final part of the assignment

Get over 55% validation accuracy on CIFAR-10 by using the layers you have implemented. You will be graded according to the following equation:

min(floor((X - 32%)) / 23%, 1) where if you get 55% or higher validation accuracy, you get full points.

In [9]:
```
# ================================================================ #
# YOUR CODE HERE:
#   Implement a FC-net that achieves at least 55% validation accuracy
#   on CIFAR-10.
# ================================================================ #

# 1. Initialize a 5-layer neural network with batchnorm and dropout enabl
hidden_dims = [500, 500, 500, 500]
model = FullyConnectedNet(hidden_dims,
                          weight_scale=0.05,
                          use_batchnorm=True,
                          dropout=0.7)

# 2. Configure the Solver using Adam optimizer
solver = Solver(model, data,
                num_epochs=15,
                batch_size=100,
                update_rule='adam',
                optim_config={
                  'learning_rate': 1e-3
                },
                lr_decay=0.95,
                verbose=True, print_every=100)

# 3. Train the model
solver.train()

# 4. Print the final validation accuracy
y_val_pred = np.argmax(model.loss(data['X_val']), axis=1)
print('Final Validation accuracy: ', np.mean(y_val_pred == data['y_val']))

# ================================================================ #
# END YOUR CODE HERE
# ================================================================ #
```

```
(Iteration 1 / 7350) loss: 2.752737
(Epoch 0 / 15) train acc: 0.099000; val_acc: 0.114000
(Iteration 101 / 7350) loss: 1.908678
(Iteration 201 / 7350) loss: 1.769681
(Iteration 301 / 7350) loss: 1.668901
(Iteration 401 / 7350) loss: 1.702253
(Epoch 1 / 15) train acc: 0.480000; val_acc: 0.450000
(Iteration 501 / 7350) loss: 1.522099
(Iteration 601 / 7350) loss: 1.603819
(Iteration 701 / 7350) loss: 1.590567
(Iteration 801 / 7350) loss: 1.549762
(Iteration 901 / 7350) loss: 1.657669
(Epoch 2 / 15) train acc: 0.517000; val_acc: 0.492000
(Iteration 1001 / 7350) loss: 1.643255
(Iteration 1101 / 7350) loss: 1.456198
(Iteration 1201 / 7350) loss: 1.773348
(Iteration 1301 / 7350) loss: 1.274098
(Iteration 1401 / 7350) loss: 1.382717
(Epoch 3 / 15) train acc: 0.538000; val_acc: 0.503000
(Iteration 1501 / 7350) loss: 1.506183
(Iteration 1601 / 7350) loss: 1.606176
(Iteration 1701 / 7350) loss: 1.296477
(Iteration 1801 / 7350) loss: 1.311204
(Iteration 1901 / 7350) loss: 1.238096
(Epoch 4 / 15) train acc: 0.565000; val_acc: 0.518000
(Iteration 2001 / 7350) loss: 1.338052
(Iteration 2101 / 7350) loss: 1.327366
(Iteration 2201 / 7350) loss: 1.335516
(Iteration 2301 / 7350) loss: 1.485323
(Iteration 2401 / 7350) loss: 1.573036
(Epoch 5 / 15) train acc: 0.534000; val_acc: 0.531000
(Iteration 2501 / 7350) loss: 1.371969
(Iteration 2601 / 7350) loss: 1.428276
(Iteration 2701 / 7350) loss: 1.189832
(Iteration 2801 / 7350) loss: 1.049776
(Iteration 2901 / 7350) loss: 1.425185
(Epoch 6 / 15) train acc: 0.602000; val_acc: 0.536000
(Iteration 3001 / 7350) loss: 1.244263
(Iteration 3101 / 7350) loss: 1.387990
(Iteration 3201 / 7350) loss: 1.285575
(Iteration 3301 / 7350) loss: 1.368796
(Iteration 3401 / 7350) loss: 1.215218
(Epoch 7 / 15) train acc: 0.620000; val_acc: 0.557000
(Iteration 3501 / 7350) loss: 1.220941
(Iteration 3601 / 7350) loss: 1.234404
(Iteration 3701 / 7350) loss: 1.121756
(Iteration 3801 / 7350) loss: 1.452042
(Iteration 3901 / 7350) loss: 1.291692
(Epoch 8 / 15) train acc: 0.636000; val_acc: 0.557000
(Iteration 4001 / 7350) loss: 1.340843
(Iteration 4101 / 7350) loss: 1.081688
(Iteration 4201 / 7350) loss: 1.230584
(Iteration 4301 / 7350) loss: 1.075483
(Iteration 4401 / 7350) loss: 1.183223
(Epoch 9 / 15) train acc: 0.624000; val_acc: 0.567000
(Iteration 4501 / 7350) loss: 1.156766
(Iteration 4601 / 7350) loss: 1.182777
(Iteration 4701 / 7350) loss: 1.167274
(Iteration 4801 / 7350) loss: 1.017182
(Epoch 10 / 15) train acc: 0.668000; val_acc: 0.573000
```

```
(Iteration 4901 / 7350) loss: 1.085028
(Iteration 5001 / 7350) loss: 1.091298
(Iteration 5101 / 7350) loss: 1.011998
(Iteration 5201 / 7350) loss: 1.101852
(Iteration 5301 / 7350) loss: 1.250214
(Epoch 11 / 15) train acc: 0.670000; val_acc: 0.586000
(Iteration 5401 / 7350) loss: 1.254231
(Iteration 5501 / 7350) loss: 1.066584
(Iteration 5601 / 7350) loss: 1.040027
(Iteration 5701 / 7350) loss: 1.034442
(Iteration 5801 / 7350) loss: 1.093837
(Epoch 12 / 15) train acc: 0.680000; val_acc: 0.569000
(Iteration 5901 / 7350) loss: 1.279081
(Iteration 6001 / 7350) loss: 0.983358
(Iteration 6101 / 7350) loss: 0.993971
(Iteration 6201 / 7350) loss: 0.994811
(Iteration 6301 / 7350) loss: 1.277861
(Epoch 13 / 15) train acc: 0.684000; val_acc: 0.587000
(Iteration 6401 / 7350) loss: 0.886297
(Iteration 6501 / 7350) loss: 0.959202
(Iteration 6601 / 7350) loss: 0.993972
(Iteration 6701 / 7350) loss: 0.985004
(Iteration 6801 / 7350) loss: 1.020248
(Epoch 14 / 15) train acc: 0.713000; val_acc: 0.585000
(Iteration 6901 / 7350) loss: 1.235162
(Iteration 7001 / 7350) loss: 0.784776
(Iteration 7101 / 7350) loss: 1.243331
(Iteration 7201 / 7350) loss: 1.008799
(Iteration 7301 / 7350) loss: 0.995209
(Epoch 15 / 15) train acc: 0.719000; val_acc: 0.578000
Final Validation accuracy:  0.589
```